

Particle Flow Filter and Differentiable Particle Filter

Haowu Duan

January 19, 2026

Contents

1	Literature Review	2
1.1	State-Space Modeling in Financial Markets: The big Picture	2
1.1.1	The gist of performing filtering	2
1.1.2	Parameter estimation	3
1.2	The Kalman Family and semi-exact methods	3
1.3	Particle based methods	3
1.3.1	Flow and kernal flow	5
1.3.2	Optimal transport resampling	5
2	Problems and experiments	7
2.1	State Space models	7
2.2	Part I	9
2.2.1	Kalman-Joseph update-condition number	9
2.2.2	KF, EKF, UKF broken by strong nonlinear model	9
2.2.3	Particle filter and particle degeneracy	11
2.2.4	EDH (LEDH) flow and filter	13
2.2.5	Matrix kernel preventing collapse	14
2.3	Part II	15
2.3.1	Stochastic flow	15
2.3.2	Soft resampling and OT resampling	15
3	A few words of reflection	16
A	Kalman Family	17
B	Particle flow and particle filter	21
B.1	Particle flow	21
B.2	Particle flow filter (kernel method)	23
B.2.1	Optimal Transport via Gradient Flow	23
B.2.2	Deriving Optimal Velocity via Kernel Restriction	23
B.2.3	Scalar Kernel	24
B.2.4	Matrix-Valued Kernel	25

1 Literature Review

1.1 State-Space Modeling in Financial Markets: The Big Picture

Financial systems are driven by latent factors that shape observable market data. Problems such as stochastic volatility, limit order book dynamics, or macroeconomic regime shifts are frequently cast as Hidden Markov Models (HMMs) or continuous state-space models (SSMs). In quantitative finance, Bayesian filtering and smoothing are key tools for recovering these hidden drivers. A central example is *stochastic volatility* modeling, where asset volatility is a drifting latent state. Recent semi-parametric methods employ particle filters to estimate it without rigid parametric forms, yielding more accurate option pricing and risk management [1]. Discrete state-space models, particularly HMMs, excel at *regime detection*. They treat market regimes (“bull” vs. “bear”, high vs. low momentum) as latent states, enabling traders to estimate real-time regime-shift probabilities and adapt allocations [2]. Filtering techniques also tackle microstructure noise in *illiquid markets*. Particle filters, for instance, reconstruct the latent “true” mid-price of sparsely traded assets (e.g., corporate bonds) from irregular, noisy transaction records [3]. As a one-dimensional demonstration, let us first write down the state-space model,

$$\begin{aligned} x_k &= f(x_{k-1}, w_k) \\ z_k &= h(x_k, u_k) \end{aligned} \tag{1}$$

where k denotes the time step, z_k is the observation, and x_k is the unobserved latent state. The noise terms w_k and u_k are mutually and temporally uncorrelated. The function f governs state evolution and h maps the state to observations. In practice we observe only z_k , which may have the same or lower dimension than x_k (the latter case is called partial observation and makes inference substantially harder due to information loss). Successful application of state-space models rests on two pillars:

1. A well-specified model with accurate parameters — traditionally this means understanding the functional forms of f and h and estimating their parameters reliably
2. An effective inference algorithm that estimates the hidden state x_k from the observation z_k — here, a filtering algorithm.

In recent years, neural networks have entered both parts of the equation. We can replace the conventional maps with learned versions: $f \rightarrow f_\theta$ and $h \rightarrow h_\theta$. Neural filtering algorithms have also emerged. We will first review the conventional numerical methods, then discuss possible ways for improvement leveraging neural networks.

1.1.1 The gist of performing filtering

Having a good model is only half the battle; we also need a reliable implementation. In research, novel filtering algorithms are developed and evaluated on synthetic data where the ground truth is known, allowing clear performance measurement. Given the state evolution f and observation model h , the goal is to estimate the hidden state x_k via the filtering distribution $p(x_k | z_{1:k})$. This is obtained recursively:

$$\begin{aligned} \textbf{Prediction:} \quad p(x_k | z_{1:k-1}) &= \int p(x_k | x_{k-1}) p(x_{k-1} | z_{1:k-1}) dx_{k-1} \\ \textbf{Update:} \quad p(x_k | z_{1:k}) &= \frac{p(z_k | x_k) p(x_k | z_{1:k-1})}{p(z_k | z_{1:k-1})} \end{aligned} \tag{2}$$

In financial applications (and most real-world settings), we do not aim to recover an exact deterministic value of x_k . Instead, the useful quantities are typically the mean m_k and covariance P_k of the filtering distribution. The filtering problem is generally a high-dimensional path integral and is intractable except in special cases (e.g., linear Gaussian SSMs). Different filtering methods (EKF, UKF, particle filters, etc.) use distinct approximations and mathematical frameworks. The key requirement for any successful filtering algorithm is that it produces numerically stable and accurate approximations to $p(x_k | z_{1:k})$ for known model dynamics.

1.1.2 Parameter estimation

Once a reliable filtering algorithm is available, it can be used to estimate the model parameters θ . We retain the functional forms of f and h but treat their parameters as variables to be learned. A good parameter set $\{\theta\}$ should assign high probability to the observed sequence. This is achieved by maximizing the marginal likelihood $p_\theta(z_{1:T})$, or equivalently minimizing the negative log-likelihood:

$$L(\theta) = - \sum_{k=1}^T \log p_\theta(z_k \mid z_{1:k-1}) \quad (3)$$

Since the logarithm is monotonically increasing, minimizing $L(\theta)$ maximizes the predictive density of each observation given past data [4]. In a Bayesian setting, where parameters are given a prior $p(\theta)$, we instead minimize,

$$L(\theta) = - \sum_{k=1}^T \log p_\theta(z_k \mid z_{1:k-1}) - \log p(\theta), \quad (4)$$

which corresponds to maximum a posteriori (MAP) estimation.

1.2 The Kalman Family and Semi-Exact Methods

For Linear-Gaussian systems, the Kalman Filter (KF) provides the optimal solution because the product of two Gaussian distributions stays Gaussian. However, financial data rarely satisfies these constraints. Extensions such as the Extended Kalman Filter (EKF) and Unscented Kalman Filter (UKF) utilize linearization and sigma-point approximations, respectively. Either way, for Gaussian distributions, we only need the mean and variance at time k , m_k and P_k .

$$\begin{aligned} x_k &= f x_{k-1} + w_k, & w_k &\sim \mathcal{N}(0, Q) \\ z_k &= h x_k + v_k, & v_k &\sim \mathcal{N}(0, R) \end{aligned} \quad (5)$$

where f and h are constant in one dimension and matrices in higher dimension. When the kernel becomes non-linear, one essentially expands it around the last input.

$$\begin{aligned} f(x_{k-1}) &\approx \left. \frac{\partial f(x)}{\partial x} \right|_{x_{k-1}} \cdot x_{k-1} = f_k x_{k-1} \\ h(x_{k-1}) &\approx \left. \frac{\partial h(x)}{\partial x} \right|_{x_k} \cdot x_k = h_k x_k \end{aligned} \quad (6)$$

This essentially introduces time dependence into the kernels but does not change the Gaussian assumption. This works fine if the higher-order terms are unimportant. UKF uses sigma points to track the mean and variance and is able to capture some higher-order correlations, but it still assumes a Gaussian distribution. Appendix A includes all the details such as key derivations. The Kalman family mostly serves as a benchmark and has very limited application. Both EKF and UKF are methods to overcome the non-linearity in the problem, with more recent progress in this direction [5]. However, there are other difficulties as well, notably the curse of dimensionality. Attempts have been made to improve in this direction, notably the Cubature Kalman Filter (CKF)[6] and Ensemble Kalman Filter (EnKF)[7, 8]. Here I do want to mention the latest progress on application of neural networks to the Kalman filter, known as KalmanNet[9, 10]. This is a method that replaces the Kalman Gain computation with a learned neural network while keeping everything else from the standard KF flow intact. The key is to train the RNN to make predictions about the Kalman gain without knowing what the noise distribution is. It gets the noise from the input data. While KalmanNet borrows design from the Kalman filter, it can be applied to non-linear, non-Gaussian state space models.

1.3 Particle-Based Methods

Particle-based methods, also known as sequential Monte Carlo, are the focus of this project. They are less model-specific, and therefore more general. There are many different particle-based methods, but the bottom line is how to represent a complicated distribution efficiently. If we were to use particles, namely samples, to represent a distribution $p(x)$, we first need to discretize the phase space.

$$p(x) = \int dy p(y) \delta(y - x) \quad (7)$$

The discretized version is,

$$p(x) \propto \sum_i \Delta p(x_i) \delta_{x, x_i} = \sum_i \omega_i \delta_{x, x_i} \quad (8)$$

where we have evenly sampled the phase space points and give them weights that realize the actual distribution. To put this in the context of sequential Monte Carlo, we demonstrate the so-called Bootstrap particle filter (BPF). At step k , we have the realization of the distribution $\{x_i^{(k-1)}, \omega_i^{(k-1)}\} \sim p(x_{k-1} | z_{1:k-1})$ as input. In the evolution/prediction step,

For $i = 1$ to N :

$$\begin{aligned} x_k^{(i)} &\sim p(x_k | x_{k-1}^{(i)}) \\ \tilde{w}_k^{(i)} &= w_{k-1}^{(i)} \quad (\text{weights unchanged in basic prediction}) \end{aligned} \quad (9)$$

In the above step, the particles simply move based on physics, but their weights stay the same. However, we need to use the observation data to score each update

For $i = 1$ to N :

$$\begin{aligned} \tilde{w}_k^{(i)} &= w_{k-1}^{(i)} \cdot p(z_k | x_k^{(i)}) \quad (\text{unnormalized importance weight}) \\ w_k^{(i)} &= \frac{\tilde{w}_k^{(i)}}{\sum_{j=1}^N \tilde{w}_k^{(j)}} \quad (\text{normalized weight}) \end{aligned} \quad (10)$$

The major problem with this simple approach is that $p(z_k | x^i)$ is most likely to be a small number. After a few steps, most of the weights will be so small due to multiplication of multiple small numbers that they become unimportant. The direct consequence is that the effective sample size of the distribution becomes small and our discretized representation becomes even more coarse-grained. This problem is known as particle weight collapse or particle degeneracy. The most straightforward way to deal with this is resampling, which is to create a different realization for the same distribution with more balanced particle samples. Resample particles with replacement according to weights to avoid degeneracy (also called particle depletion or weight collapse). High-weight particles get duplicated; low-weight particles die out. This step is typically performed when $N_{\text{eff}} < N_{\text{threshold}}$ (e.g., $N_{\text{threshold}} = N/2$). The most straightforward resampling method is multinomial resampling, namely, generating N independent samples from the categorical distribution:

$$\mathbf{x}_t^{(i)} \sim \text{Categorical}(\{w_t^{(j)}\}_{j=1}^N), \quad i = 1, \dots, N \quad (11)$$

where particle $\mathbf{x}_t^{(j)}$ is selected with probability $w_t^{(j)}$. After resampling, set all weights to $w_t^{(i)} = 1/N$ for $i = 1, \dots, N$. In the code, we will use systematic resampling, which is more efficient. For each sample we generate a single uniform random number $u \sim \mathcal{U}(0, 1/N)$ and define:

$$u^{(i)} = u + \frac{i-1}{N}, \quad i = 1, \dots, N \quad (12)$$

Then select particle indices j_i such that:

$$\sum_{k=1}^{j_i-1} w_t^{(k)} < u^{(i)} \leq \sum_{k=1}^{j_i} w_t^{(k)} \quad (13)$$

Set $\mathbf{x}_t^{(i)} = \mathbf{x}_t^{(j_i)}$ and $w_t^{(i)} = 1/N$ for all i . The particles rely on the duplicates or multiplicity to mimic the underlying distribution. As a consequence, we have a sample impoverishment problem. Namely, to have more particles with non-trivial weights, we have to duplicate large-weight particles while killing the small-weight particles. After resampling, we set all particles to have equal weight and continue the process.

Because all particles have equal weight, this means the weights carry no information about the distribution and we rely on multiplicity to represent the distribution right after each resampling. This is extremely inefficient and hints that using multiplicity should have been our first choice. Then the problem is very similar to numerical procedures to solve stochastic differential equations. Therefore, the prediction step stays the same because it doesn't touch the distribution, just changes the basis; it is the update step that is responsible for correcting the particles. Because there are no diverse weights anymore, the only way to do this is to move the particles again. This is what particle flow particle filter does. These are observation-guided evolution equations. When it comes to how, one has a good deal of freedom.

1.3.1 Flow and Kernel Flow

Alternatively, we don't use weights to represent the distribution at all, namely setting the weights to be uniform. In the update step, we are still using the dynamics to evolve the particles, but in the update step, instead, we move the particles again based on the observation so that the "corrected" distribution is a better realization of the target probability. We will discuss four different ways to move particles.

1. **Exact Flow:** Exact Daum-Huang (EDH) flow and Local Exact Daum-Huang (LEDH) flow [11, 12, 13] use deterministic PDEs to move the particles from prior to posterior. In practice, the algorithms use EKF or UKF to move the filter forward. However, this is not the same as assuming the distribution is Gaussian. If the particles are sampled from a non-Gaussian distribution, then after the Gaussian update, the distribution will still be non-Gaussian. The upside of the Gaussian assumption is that we avoid computing the score $\nabla \ln p$. However, the downside is that strongly nonlinear problems will break the flow.
2. **Stochastic Flow** [14] Conceptually, the most straightforward way to generalize ODE is SDE. We still make Gaussian assumptions about the flow kernel but use stochastic differential equations to do the evolution. When it comes to stiff equations, the paper [15] proposed using a differential equation to determine the optimal flow schedule instead of a straight line.
3. **Invertible Particle Flow-Particle Filter:** Particle flow equations have discretization errors, and they don't really transform the particles to the target probability. Think of the flow as a coordinate transformation; not only should you transform the map, but also account for the Jacobian. PF-PF re-introduces the weights back into the picture and uses the weight update to make adjustments to compensate for the effect of this Jacobian. Because the Jacobian is flow-specific, if the flow equation kernel is identical across particles, as in the case of EDH, then this Jacobian will make no contribution and will be canceled by normalization. In the case of particle-specific evolution, this Jacobian is highly non-trivial and will be important.
4. **Kernel-Embedded Particle Flow Filter in an RKHS:** [16, 17]: Instead of moving particles with a certain schedule, we move the particles along the trajectory where the difference between the particle distribution and the target is decreasing as fast as possible. Define target posterior $\pi(x) := p(x_t | z_{1:t})$ and intermediate density $q_\lambda(x)$ at pseudo-time $\lambda \in [0, 1]$. With $q_0 = p(x_t | z_{1:t-1})$ (prior), $q_1 = \pi$ (posterior).

$$\frac{dx_\lambda}{d\lambda} = v_\lambda(x_\lambda) \quad (14)$$

Choose velocity v_λ to minimize KL divergence as fast as possible:

$$D_{KL}(q_\lambda \| \pi) = \int q_\lambda(x) \log \frac{q_\lambda(x)}{\pi(x)} dx \quad (15)$$

we need to formulate the above equation into an optimization problem. However, the intuition of this method is clear: you want to evolve the point along a "straight" direction to the desired point. On the implementation side,

1.3.2 Optimal Transport Resampling

Previously, we have discussed the sample impoverishment problem introduced by low-quality resampling. Therefore, we need to think about how to improve it. Before and after resampling, we are preserving the distribution or at least trying to. We need to come up with the best way to do it. In [18], the author proposed using optimal transport to do the resampling. An easy way to see this is that, before the resampling, we have $\{\omega_i, x_i\}$, and after the resampling, we have $\{\tilde{\omega}_i, \tilde{x}_i\}$. If we think of x_i as basis vectors and ω_i as the coefficients, we are dealing with a linear transformation problem. The transformation will preserve the vector itself, or preserve the mean at least. One should not take this analogy too far. The key benefit is that OT resampling is deterministic instead of stochastic; this will avoid the sampling noise introduced by systematic resampling, therefore removing its contribution to the variance of the particles, namely increasing the quality of covariance estimation. However, this method limits the use of neural network architecture in the filtering algorithm. This method is not differentiable because it relies on solving a discrete Linear Programming (LP) problem to find the optimal transport plan. One just uses LP solvers like the Simplex algorithm or Interior Point methods. These algorithms don't need derivatives of the optimal solution with respect to inputs—they only need the cost matrix (particle distances) and

constraints (matching weights). They work by walking along edges of the polytope or navigating through its interior. The key innovation of [19] is to introduce entropy regularization so that the optimal coupling P^* is differentiable as a function of the cost matrix, therefore turning the whole resampling operation differentiable. The following is the loss function for the transport cost,

$$W_{2,\epsilon}^2(\alpha_N, \beta_N) = \min_{P \in S(a,b)} \sum_{i,j=1}^N p_{i,j} \left(c_{i,j} + \epsilon \log \frac{p_{i,j}}{a_i b_j} \right) \quad (16)$$

Another key innovation is that by adding the entropy term $\epsilon \log \frac{p_{i,j}}{a_i b_j}$ instead of $\epsilon \log p_{i,j}$, the author made it possible to determine the transport plan from the result of the dual problem, which is $\mathcal{O}(N)$, instead of $\mathcal{O}(N^2)$ like [18], which opens the possibility for modeling more complicated systems with large numbers of particles.

More recently, people have also used neural networks to solve the Monge problem itself [20]. GradNetOT exploits Brenier’s theorem, which says that under certain conditions (continuous densities, squared Euclidean cost), the Monge and Kantorovich problems are equivalent, and the unique transport map $T(x)$ is the gradient of a convex function $\phi(x)$, where $\phi(x)$ satisfies the Monge-Ampère equation

$$p(x) = q(\nabla\phi(x)) \det(H_\phi(x)) \quad (17)$$

where $H_\phi = J_T$ is the $N \times N$ positive semi-definite Hessian of ϕ . However, this architecture is not directly applicable to resampling because the transformation will push the continuous distribution $p(x)$ to the target distribution $q(y)$. We will discuss the possible ways to make it work.

In addition, there are of course alternatives to OT resampling. A notable example is Particle Transformer[21, 22], where we replace the OT resampler with a Transformer mechanism (self-attention) to allow particles to interact with each other and re-weight them differentially. To make this review more focused, we will not discuss it further. The key idea is that you require the transformer to output particles with equal weights. Therefore, it learns from the data to put particles into the right positions.

Optimal transport resampling outperforms in certain scenarios, but its real power lies in that it makes the entire filtering architecture differentiable. One can easily use neural networks to model the dynamics or observation, which has been extensively discussed in [23]. I think there is no point in making a comprehensive review here as it is problem-specific and requires careful testing; there is no one-fits-all solution. This field is exploding and there are so many different architectures that one can try. As pointed out in Bonus Question 2, it would be nice if we didn’t have to solve the optimal transport problem repeatedly, because each resampling is a different set of initial particles and weights. One way to deal with this problem is amortized optimization[24], where you train a model to predict solutions to optimization problems rather than solving each problem from scratch. Its application to the OT problem is known as Meta-OT [25]. The idea is actually very easy: when we run a filtering algorithm, we need to solve the resampling many times. Each time, we will have a transport schedule from the dual problem. We simply run the filter many times and record what kind of transport plan we will run into in such problems. Then we train a neural network to make predictions.

2 Problems and experiments

2.1 State Space models

We first list all the models that I have used in the numerical experiments.

1. Linear-Gaussian Model

Dynamics:

$$X_n = F \cdot X_{n-1} + B \cdot V_n, \quad V_n \sim \mathcal{N}(0, I)$$

Observation:

$$Y_n = H \cdot X_n + D \cdot W_n, \quad W_n \sim \mathcal{N}(0, I)$$

Parameters:

- $F \in \mathbb{R}^{n_x \times n_x}$: State transition matrix
- $B \in \mathbb{R}^{n_x \times n_v}$: Process noise matrix
- $H \in \mathbb{R}^{n_y \times n_x}$: Observation matrix
- $D \in \mathbb{R}^{n_y \times n_w}$: Observation noise matrix
- $Q = BB^T$: Process noise covariance
- $R = DD^T$: Observation noise covariance

2. Acoustic Tracking Full Model (4 targets, 25 sensors)

Dynamics (4 independent targets):

$$X_{t+1} = FX_t + w_t, \quad w_t \sim \mathcal{N}(0, V)$$

where $F \in \mathbb{R}^{16 \times 16}$ is block diagonal with 4 identical blocks, $X \in \mathbb{R}^{16}$

Observation (additive amplitude):

$$z_s = \sum_{i=1}^4 \frac{\Psi}{r_{s,i}^2 + d_0} + \epsilon_s, \quad \epsilon_s \sim \mathcal{N}(0, \sigma_w^2)$$

Parameters:

- $V_{\text{true}} = \frac{1}{20} \text{diag}(V_{\text{single}}, \dots, V_{\text{single}})$ (4 blocks)
- $V_{\text{filter}} = \text{diag}\left(\begin{bmatrix} 3 & 0 & 0.1 & 0 \\ 0 & 3 & 0 & 0.1 \\ 0.1 & 0 & 0.03 & 0 \\ 0 & 0.1 & 0 & 0.03 \end{bmatrix}, \dots\right)$ (4 blocks)
- $\Psi = 10, d_0 = 0.1, \sigma_w = 0.1$
- Sensors: 5×5 grid on $[0, 40] \times [0, 40]$

3. Range-Bearing Model

Dynamics:

$$X_t = FX_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q), \quad X = [x, y]^T$$

Observation (nonlinear):

$$\begin{bmatrix} \text{range} \\ \text{bearing} \end{bmatrix} = \begin{bmatrix} \sqrt{(x - x_s)^2 + (y - y_s)^2} \\ \arctan 2(y - y_s, x - x_s) \end{bmatrix} + \begin{bmatrix} v_r \\ v_b \end{bmatrix}$$

where $v_r \sim \mathcal{N}(0, \sigma_r^2), v_b \sim \mathcal{N}(0, \sigma_b^2)$

Parameters:

- F : State transition (default: identity)
- Q : Process noise covariance (default: $0.01I_2$)

- (x_s, y_s) : Sensor position (default: origin)
- $\sigma_r = 0.1$: Range noise std
- $\sigma_b = 0.01$: Bearing noise std (radians)

4. Two-Sensor Bearing-Only Model

Dynamics (static):

$$X_{t+1} = X_t, \quad X = [x, y]^T$$

Observation:

$$\begin{bmatrix} \text{bearing}_1 \\ \text{bearing}_2 \end{bmatrix} = \begin{bmatrix} \arctan 2(y - y_{s,1}, x - x_{s,1}) \\ \arctan 2(y - y_{s,2}, x - x_{s,2}) \end{bmatrix} + \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

where $v_i \sim \mathcal{N}(0, \sigma_b^2)$

Parameters:

- Prior: $\mu_0 = [3.0, 5.0]^T, \Sigma_0 = \begin{bmatrix} 1000 & 0 \\ 0 & 2 \end{bmatrix}$ (highly anisotropic)
- Sensors: $[(3.5, 0), (-3.5, 0)]$
- $\sigma_b = 0.2$: Bearing noise std
- $R = \text{diag}(0.04, 0.04)$

5. Stochastic Volatility Model

Dynamics (linear):

$$x_t = \alpha x_{t-1} + \sigma w_t, \quad w_t \sim \mathcal{N}(0, 1)$$

Observation (nonlinear, non-Gaussian):

$$y_t = \beta \exp(x_t/2) v_t, \quad v_t \sim \mathcal{N}(0, 1)$$

Parameters:

- $\alpha = 0.91$: Persistence parameter
- $\sigma = 1.0$: Volatility of volatility
- $\beta = 0.5$: Scale parameter
- Stationary variance: $\sigma^2/(1 - \alpha^2)$

6. Lorenz 96 Model

Dynamics (chaotic, high-dimensional):

$$\frac{dx_i}{dt} = (x_{i+1} - x_{i-2})x_{i-1} - x_i + F$$

with cyclic boundary conditions, integrated using 4th-order Runge-Kutta

Observation (linear, sparse):

$$Y = HX + \epsilon, \quad \epsilon \sim \mathcal{N}(0, R)$$

where H observes every 4th variable

Parameters:

- $n_x = 1000$: State dimension
- $F = 8.0$: Forcing constant (chaotic regime)
- $\Delta t = 0.05$: Integration time step
- Observation interval: every 20 RK4 steps
- $\sigma_{\text{obs}} = 1.0$: Observation noise std
- $R = I_{n_y}$, where $n_y = 250$ (25% observed)

2.2 Part I

2.2.1 Kalman-Joseph update-condition number

The first topic we investigate is Joseph update and condition number. The following table is the calculation based on the 2-d linear Gaussian state space model with parameters:

$$\begin{aligned} F &= \begin{pmatrix} 0.95 & 0 \\ 0 & 0.95 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \\ H &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad D = \begin{pmatrix} 10^{-8} & 0 \\ 0 & 10^{-8} \end{pmatrix} \\ \mu_0 &= \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \Sigma_0 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \end{aligned}$$

where the observation noise is of the order 10^{-8} , which leads to $R \sim 10^{-16}$.

Table 1: Comparison of Kalman Filter with Joseph Form vs. Standard Update

Metric	Joseph Form	Standard Update
RMSE	9.708×10^{-9}	9.708×10^{-9}
Log-Likelihood	-566.840	-566.840
Performance Metrics		
Wall Time (s)	0.0203	0.0206
Time per Timestep (ms)	0.1016	0.1030
Peak Memory (MB)	0.274	0.273
Memory Increase (MB)	0.438	0.531
Numerical Stability		
Condition Number	1.0 (all timesteps)	∞ (all timesteps)

Small observation noise will lead to numerical instability and an extremely large condition number in the updated covariance. In higher dimensions, small noise is not the only such scenario, but the argument for why the Joseph form will restore numerical stability stays the same. Let us first take a look at what small R in 1-D looks like. With,

$$\lim_{R \rightarrow 0} K = \lim_{R \rightarrow 0} \frac{P^- h}{h^2 P^- + R} = \frac{1}{h} \quad (18)$$

With the standard update: $P^+ = (1 - Kh)P^- \approx 0$. However, with the Joseph form,

$$P^+ = (I - Kh)P^-(I - Kh) + K^2 R \sim \frac{R}{h^2} \quad (19)$$

we see that $\frac{R}{h^2}$ basically has condition number one because the noise scale is an overall factor. This will legitimately restore numerical stability in higher dimensions due to $K^2 R \rightarrow K R K^T$. Let $H = I$, $R = \epsilon I$, $P^- = I$. Then $K \approx I$. The condition number for the residue is one.

2.2.2 KF, EKF, UKF Broken by Strongly Nonlinear Model

We first use the Stochastic Volatility model,

$$x_{t+1} = 0.91x_t + w_t, \quad w_t \sim \mathcal{N}(0, 1) \quad (20)$$

$$y_t = 0.5 \exp(x_t/2) \cdot v_t, \quad v_t \sim \mathcal{N}(0, 1) \quad (21)$$

with the initial condition drawn from $N(0, \frac{\sigma^2}{1-\alpha^2})$, where $\alpha = 0.91, \sigma = 1$. The result shows that these two filters, though designed to handle non-linear problems, are completely broken by the Stochastic Volatility model. This is due to the fact that when trying to linearize the observation model, the noise term ensures that the mean is always zero. Even though we gave the filter an initial kick, it eventually converged to zero mean. Another issue is that the variance received zero update due to zero h . In addition, I also did an experiment on the range-bearing model,

State Space: $\mathbf{x}_t = [x_t, y_t]^T \in \mathbb{R}^2$ (2D position)

State Evolution:

$$\mathbf{x}_t = F\mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(0, Q) \quad (22)$$

where $F = I_2$ (constant position model) and $Q = 0.01 \cdot I_2$. The Observation Model is,

$$\mathbf{y}_t = h(\mathbf{x}_t) + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(0, R) \quad (23)$$

where the nonlinear observation function is:

$$h(\mathbf{x}_t) = \begin{bmatrix} \text{range}_t \\ \text{bearing}_t \end{bmatrix} = \begin{bmatrix} \sqrt{(x_t - x_s)^2 + (y_t - y_s)^2} \\ \arctan 2(y_t - y_s, x_t - x_s) \end{bmatrix} \quad (24)$$

Parameters:

- Sensor position: $\mathbf{x}_s = [x_s, y_s]^\top = [0, 0]^\top$
- Observation noise: $R = \text{diag}(\sigma_{\text{range}}^2, \sigma_{\text{bearing}}^2)$
 - Range noise: $\sigma_{\text{range}} = 0.1$
 - Bearing noise: $\sigma_{\text{bearing}} = 0.01$ radians
- Initial state: $\mathbf{x}_0 \sim \mathcal{N}(\boldsymbol{\mu}_0, \Sigma_0)$ where $\boldsymbol{\mu}_0 = [1, 1]^\top$, $\Sigma_0 = I_2$

Jacobian: For the Extended Kalman Filter, the observation Jacobian is:

$$H(\mathbf{x}_t) = \frac{\partial h}{\partial \mathbf{x}} \bigg|_{\mathbf{x}_t} = \begin{bmatrix} \frac{x_t - x_s}{r_t} & \frac{y_t - y_s}{r_t} \\ -\frac{y_t - y_s}{r_t^2} & \frac{x_t - x_s}{r_t^2} \end{bmatrix} \quad (25)$$

where $r_t = \sqrt{(x_t - x_s)^2 + (y_t - y_s)^2}$ is the range.

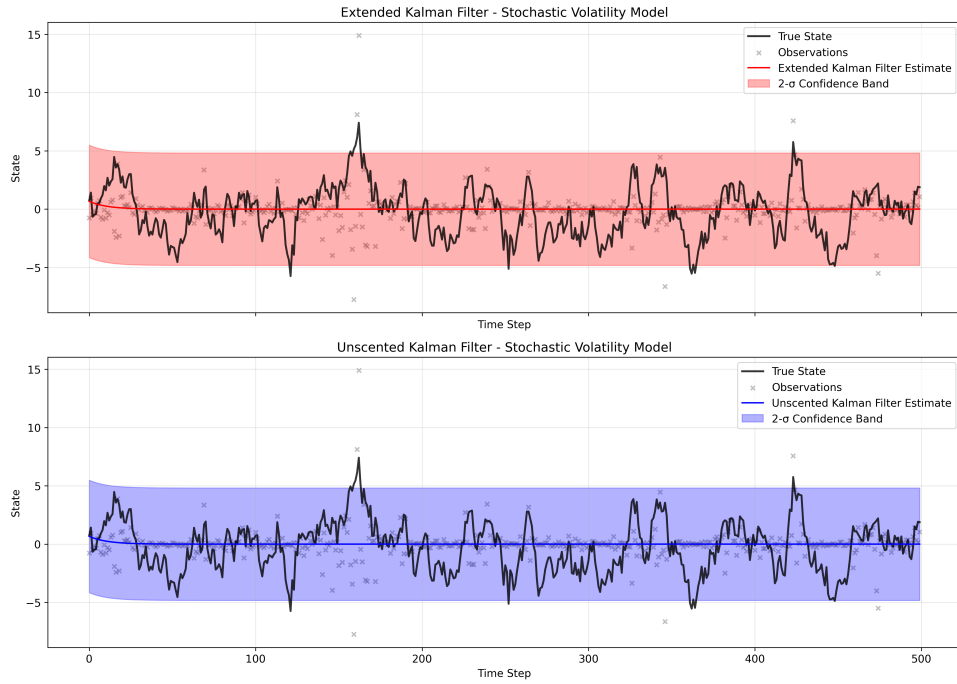


Figure 1: EKF and UKF result on stochastic volatility model

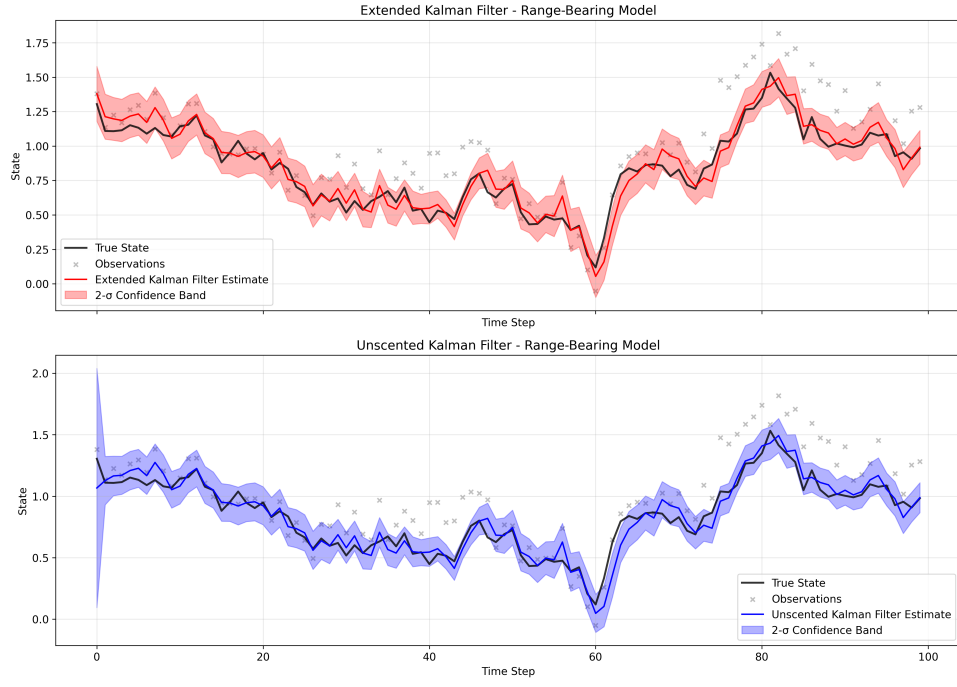


Figure 2: EKF and UKF result on range bearing model

we see that the range-bearing model works with UKF and EKF because we can get a reasonable Jacobian from the observation model.

2.2.3 Particle Filter and Particle Degeneracy

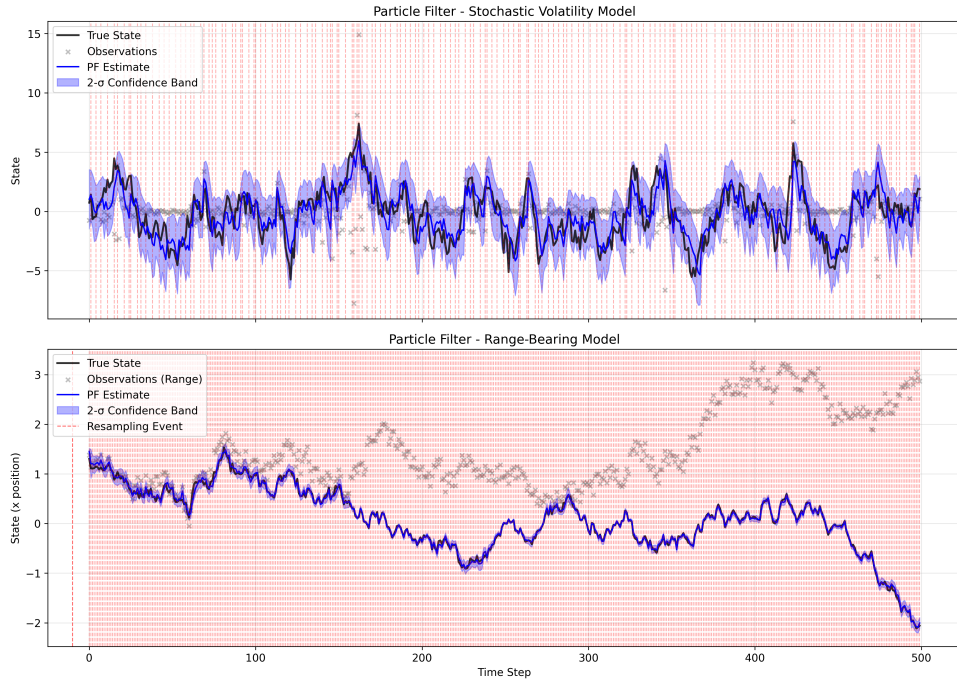


Figure 3: We see that particle filter tracking the means of both models pretty well

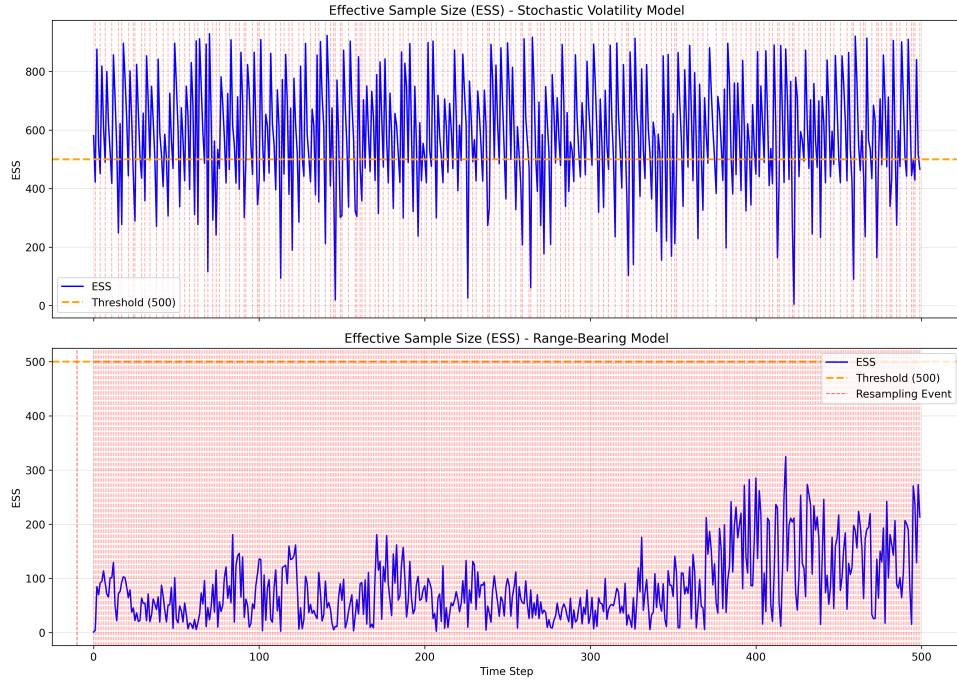


Figure 4: Effective sample size a function of time

The red lines are the times where we did resampling. It turns out that the range-bearing model had very bad initialization, but the particle filter somehow managed to increase sample size at later times. This is likely due to the fact that particles were moved into the right position, so even with very few particles, it can track the distribution very well. However, a depleted sample size means that uncertainty estimation is unreliable.

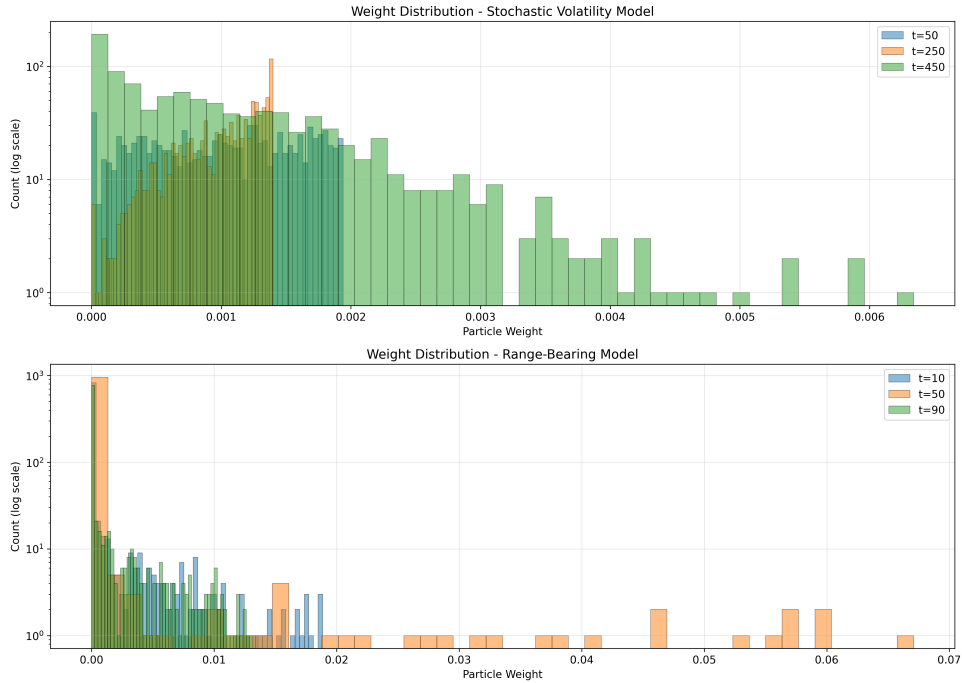


Figure 5: Weight distribution at three different times. It further demonstrates that bad initialization for the range-bearing model had collapsed weight distribution from the very beginning

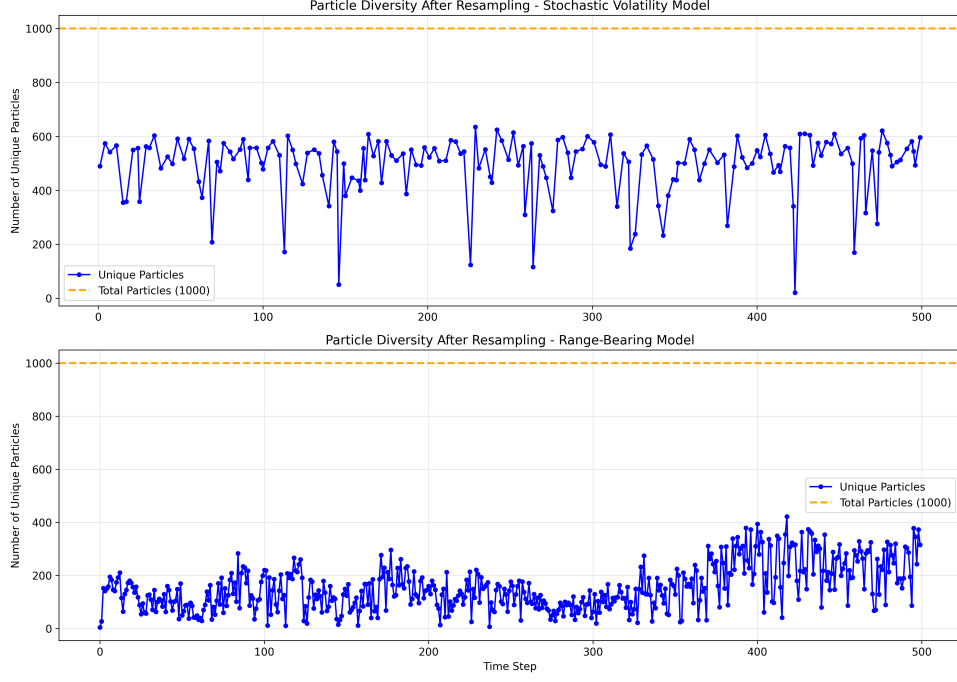


Figure 6: Particle diversity for the Stochastic Volatility model is reasonable.

2.2.4 EDH (LEDH) Flow and Filter

My flow filter and invertible flow particle filter did not work for the acoustic tracking model or even the simplified acoustic tracking model. I could not reproduce the result from [26]. However, it works for the range-bearing model, so I include the performance for the range-bearing model here. It is similar to Table 1 from [26]. The performance of PF is the baseline, and all the flow and invertible filters outperform the Bootstrap particle filter. In addition, LEDH invertible has the best performance in terms of accuracy, but it is the slowest among all the filters.

Table 2: Particle Filter Comparison on Range-Bearing Model

Filter	RMSE	Log-Lik	Wall Time (s)	Time/Step (ms)	Peak Mem (MB)
PF	0.0610	745.41	50.81	101.63	16.05
EDH Flow	0.0593	—	7.26	72.55	1.48
EDH Invertible	0.0584	423.50	22.15	221.48	4.64
LEDH Flow	0.0584	—	228.09	2280.95	1.46
LEDH Invertible	0.0582	428.97	163.06	1630.56	1.67

Filter	Particles	Resample Rate	Timesteps
PF	1000	1.00	500
EDH Flow	500	—	100
EDH Invertible	2000	0.45	100
LEDH Flow	500	—	100
LEDH Invertible	500	0.37	100

2.2.5 Matrix Kernel Preventing Collapse

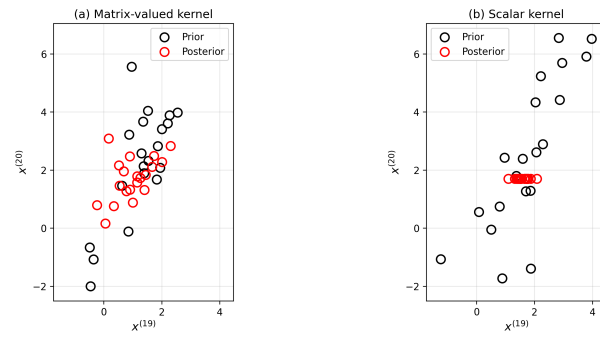


Figure 7: We can see clearly that by switching to a matrix kernel, x_{20} no longer collapses into one point

2.3 Part II

2.3.1 Stochastic Flow

My stochastic flow filter is not working well even for the Kalman filter, and I have not yet been able to find out what is going on. It only works when the noise is set to zero.

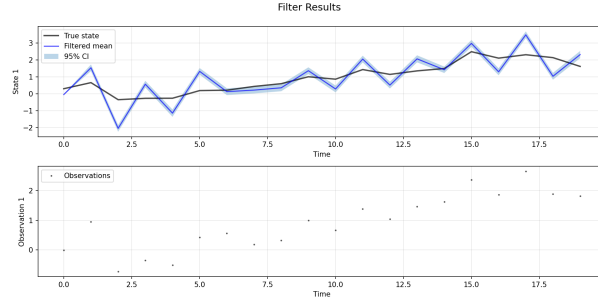


Figure 8: The current version of SDE filter, when diffusion scale is set to 0.001

2.3.2 Soft Resampling and OT Resampling

Table 3: Resampling Method Comparison on Stochastic Volatility Model

Method	Param	RMSE	Log-Lik	Wall Time (s)	Time/Step (ms)	Resample Rate
<i>Baseline</i>						
PF (Systematic)	—	1.182	−436.29	3.65	7.31	0.356
<i>Optimal Transport (OT) Resampling</i>						
OT	$\epsilon = 0.1$	NaN	NaN	32.61	65.23	0.290
OT	$\epsilon = 0.3$	1.372	−477.29	25.04	50.08	0.390
OT	$\epsilon = 0.5$	1.379	−476.44	18.32	36.65	0.390
OT	$\epsilon = 1.0$	NaN	NaN	5.92	11.85	0.286
<i>Soft Resampling</i>						
Soft	$\alpha = 0.5$	1.173	−436.42	1.26	2.52	0.534
Soft	$\alpha = 0.7$	1.167	−436.04	1.25	2.51	0.420
Soft	$\alpha = 0.9$	1.175	−435.56	1.27	2.54	0.368

Here I only have the results at their face value, so I will not speculate why the results turned out this way.

3 A Few Words of Reflection

This is a well-structured test, and I learned a lot from doing the background research, especially the knowledge I gained about Optimal Transport, which might actually turn out to be useful in the future. I now intend to learn about other applications of filtering and experiment with them more.

I didn't completely finish the test. I only included the experiments that directly relate to the question, even though I ran quite a few others. I simply ran out of time to write up the results properly. I appreciate the experience. It made me realize that I need to manage my time better when navigating multiple commitments. I put too much focus on reading background material and exploring ideas for using neural networks to improve filtering. In the end, I didn't have enough time left to debug the code, complete the experiments, or write tests that would actually catch numerical instability. During my academic years, I never had to write formal tests for my code, so this was by far the biggest challenge I've faced. My original plan was to first make things work, then go back and add tests to check the edge cases. Maybe that approach just doesn't work well when I'm adding tests after the fact. I will continue working on this project and force myself to write proper tests to debug the invertible flow code. Right now, I still cannot reproduce the results from [26], because the acoustic tracking model breaks all of my EDH flow, EDH invertible, and their local counterparts.

Thank you for giving me this opportunity to do the test!

A Kalman Family

Notation:

- $\mathbf{m}_t^-$ = *a priori* state estimate (mean of the predicted distribution)
- $\mathbf{m}_t$ = *a posteriori* state estimate (mean of the updated distribution)
- $\mathbf{P}_t^-$ = *a priori* error covariance
- $\mathbf{P}_t$ = *a posteriori* error covariance
- $p(x_k|z_{1:k-1})$ = prior distribution
- $p(x_k|z_{1:k})$ = posterior Distribution

we use the following initial condition,

$$p(x_0) = \mathcal{N}(x_0|m_0, P_0) \quad (26)$$

For the Kalman filter, the noise and operators are both Gaussian, so the distributions stay Gaussian at all times. All we need to do is to keep track of the mean and variance of the prior distribution $p(x_k|y_{1:k})$. In the predict step, the goal is to compute $p(x_t|z_{1:t-1})$ from $p(x_{t-1}|z_{1:t-1})$. Given $p(x_{t-1}|z_{1:t-1}) = \mathcal{N}(x_{t-1}|m_{t-1}, P_{t-1})$, we have:

$$p(x_t|z_{1:t-1}) = \int dx_{t-1} p(x_t|x_{t-1}) p(x_{t-1}|z_{1:t-1}) \quad (27)$$

From the state evolution model:

$$p(x_t|x_{t-1}) = \mathcal{N}(x_t|f x_{t-1}, Q) \quad (28)$$

predicted/evolved mean is,

$$m_t^- = \mathbb{E}[x_t|z_{1:t-1}] \quad (29)$$

$$= f m_{t-1} \quad (30)$$

predicted/evolved variance

$$P_t^- = \text{Var}[x_t|y_{1:t-1}] \quad (31)$$

$$= Q + f^2 P_{t-1} \quad (32)$$

from the predict step, we get,

$$p(x_t|z_{1:t-1}) = \mathcal{N}(x_t|m_t^-, P_t^-) \quad (33)$$

where

$$\boxed{m_t^- = a m_{t-1}, \quad P_t^- = a^2 P_{t-1} + Q} \quad (34)$$

In the update step, we need to incorporate observation z_t using Bayes' rule.

$$p(x_t|z_{1:t}) = \frac{p(z_t|x_t) p(x_t|z_{1:t-1})}{p(z_t|z_{1:t-1})} \quad (35)$$

From the observation model:

$$p(z_t|x_t) = \mathcal{N}(z_t|h x_t, R) \quad (36)$$

We need to compute the product of two Gaussians:

$$p(x_t|z_{1:t}) \propto \mathcal{N}(z_t|h x_t, R) \cdot \mathcal{N}(x_t|m_t^-, P_t^-) \quad (37)$$

$$\propto \exp\left(-\frac{1}{2} \frac{(z_t - h x_t)^2}{R}\right) \exp\left(-\frac{1}{2} \frac{(x_t - m_t^-)^2}{P_t^-}\right) \quad (38)$$

Expanding the exponents:

$$-\frac{1}{2} \left[\frac{(z_t - h x_t)^2}{R} + \frac{(x_t - m_t^-)^2}{P_t^-} \right] = -\frac{1}{2} \left[\frac{z_t^2 - 2 z_t h x_t + h^2 x_t^2}{R} + \frac{x_t^2 - 2 x_t m_t^- + (m_t^-)^2}{P_t^-} \right] \quad (39)$$

Collecting terms in x_t^2 and x_t : $-\frac{1}{2} \left[\left(\frac{h^2}{R} + \frac{1}{P_t^-} \right) x_t^2 - 2 \left(\frac{hz_t}{R} + \frac{m_t^-}{P_t^-} \right) x_t + \text{const} \right]$. This is a Gaussian in x_t . Completing the square, we have $\frac{1}{P_t} = \frac{h^2}{R} + \frac{1}{P_t^-}$. P is the uncertainty, so the inverse of P is like precision; by adding information about the measurement $\frac{h^2}{R}$, we get additional precision, which is exactly what we are aiming for. When $\frac{h^2}{R} = 0$, namely large noise from observation, we say we can't trust the measurement and we should stick with the dynamics. We see that both mean and variance will receive no update. On the other hand, if $\frac{h^2}{R} \ll \frac{1}{P_t^-}$, then the observation will carry more weight and the result from dynamics should be disregarded. The updated variance is

$$P_t = \frac{RP_t^-}{h^2P_t^- + R} \quad (40)$$

For simplicity, we define the **Kalman gain**:

$$K_t = \frac{P_t^- h}{h^2P_t^- + R} \quad (41)$$

Then:

$$P_t = (1 - K_t h) P_t^- \quad (42)$$

After some algebra:

$$m_t = m_t^- + K_t(z_t - hm_t^-) \quad (43)$$

where we denote the innovation $v_t = z_t - hm_t^-$, namely the difference between the real observation and the predicted observation from prior mean. In summary, we get,

$$\boxed{\begin{aligned} K_t &= \frac{P_t^- h}{h^2P_t^- + R} \\ v_t &= y_t - hm_t^- m_t = m_t^- + K_t v_t \\ P_t &= (1 - K_t h) P_t^- \end{aligned}} \quad (44)$$

At this point, we can already see that what the filter does is to correct the distribution based on the observed data point. Essentially, we want the mean and covariance from the updated distribution at each time step. For higher dimensions, all we have to do is to promote the scalar recursive relation into matrices. **Prediction Step:**

$$\hat{\mathbf{x}}_t^- = \hat{\mathbf{F}} \hat{\mathbf{x}}_{t-1}^+ \quad (45)$$

$$\mathbf{P}_t^- = \hat{\mathbf{F}} \mathbf{P}_{t-1}^+ \hat{\mathbf{F}}^T + \mathbf{Q} \quad (46)$$

Correction Step:

$$\hat{\mathbf{K}}_t = \mathbf{P}_t^- \mathbf{H}^T [\mathbf{H} \mathbf{P}_t^- \mathbf{H}^T + \mathbf{R}]^{-1} \quad (\text{Kalman gain}) \quad (47)$$

$$\hat{\mathbf{x}}_t^+ = \hat{\mathbf{x}}_t^- + \hat{\mathbf{K}}_t(\mathbf{z}_t - \mathbf{H} \hat{\mathbf{x}}_t^-) \quad (48)$$

$$\hat{\mathbf{P}}_t^+ = (\mathbf{I} - \hat{\mathbf{K}}_t \mathbf{H}) \mathbf{P}_t^- \quad (49)$$

We know that the Kalman filter requires linear models and Gaussian noise. However, this is very restrictive. For cases where the noises are Gaussian but models are non-linear, we have the Extended Kalman filter (EKF) and Unscented Kalman Filter (UKF). The idea for the extension is to somehow make approximations so that we can use the Kalman filter. EKF linearizes the models via first-order Taylor expansion around the current estimate.

$$\mathbf{F}_t = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_{t-1}^+}, \quad \mathbf{H}_t = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_t^-} \quad (50)$$

Apply Kalman filter equations using time-varying $\mathbf{F}_t, \mathbf{H}_t$. Evolve using time-dependent operators,

$$\hat{\mathbf{x}}_t^- = f(\hat{\mathbf{x}}_{t-1}^+) \quad (51)$$

$$\mathbf{P}_t^- = \mathbf{F}_t \mathbf{P}_{t-1}^+ \mathbf{F}_t^T + \mathbf{Q} \quad (52)$$

Update using $\mathbf{H}_t$ and predicted measurement $\hat{\mathbf{z}}_t^- = h(\hat{\mathbf{x}}_t^-)$.

$$\hat{\mathbf{K}}_t = \mathbf{P}_t^- \mathbf{H}_t^T [\mathbf{H}_t \mathbf{P}_t^- \mathbf{H}_t^T + \mathbf{R}]^{-1} \quad (\text{Kalman gain}) \quad (53)$$

$$\hat{\mathbf{x}}_t^+ = \hat{\mathbf{x}}_t^- + \mathbf{K}_t (\mathbf{z}_t - \mathbf{H}_t \hat{\mathbf{x}}_t^-) \quad (54)$$

$$\hat{\mathbf{P}}_t^+ = (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t) \mathbf{P}_t^- \quad (55)$$

However, there are strong limitations to this approach, namely: only first-order accurate; can diverge for strong nonlinearity; requires Jacobian computation (may be expensive or analytically intractable); assumes Gaussian noises. As for the UKF, it propagates uncertainty through nonlinear functions using **sigma points** (also called unscented points) - deterministically chosen samples that capture mean and covariance. Here we further introduce some definitions:

- $\mathbf{P}_{zz,t}^-$ = predicted measurement covariance
- $\mathbf{P}_{xz,t}^-$ = cross-covariance between state and measurement

This is because Kalman gain has a more general form

$$\mathbf{K}_t = \mathbf{P}_{xz,t}^- (\mathbf{P}_{zz,t}^-)^{-1} \quad (56)$$

where

- $\mathbf{P}_{xz,t}^- = \text{Cov}(\mathbf{x}_t - \hat{\mathbf{x}}_t^-, \mathbf{z}_t - \hat{\mathbf{z}}_t^-)$ (state-innovation cross-covariance)
- $\mathbf{P}_{zz,t}^- = \text{Cov}(\mathbf{z}_t - \hat{\mathbf{z}}_t^-)$ (innovation covariance)

This expression is exact for the linear-Gaussian case and approximate (but very effective) for nonlinear cases when the covariances are estimated via linearization (EKF) or sigma points (UKF). Associated with the sigma points are two sets of weights used for computing the mean and covariance after nonlinear transformation:

$$W_0^{(m)} = \frac{\lambda}{n + \lambda} \quad (57)$$

$$W_0^{(c)} = \frac{\lambda}{n + \lambda} + (1 - \alpha^2 + \beta) \quad (58)$$

$$W_i^{(m)} = W_i^{(c)} = \frac{1}{2(n + \lambda)}, \quad i = 1, \dots, 2n \quad (59)$$

where

- $W_i^{(m)}$ are used to recover the transformed **mean**,
- $W_i^{(c)}$ are used to recover the transformed **covariance**,
- β is typically set to 2 (optimal for Gaussian distributions).
- α - Spread Control Parameter, we set it to 10^{-3}
- κ - Secondary Scaling Parameter, we set it to zero.
- $\lambda = \alpha^2(n_x + \kappa) - n_x$

Prediction Step (Time Update):

Generate $2n_x + 1$ sigma points from $\hat{\mathbf{x}}_{t-1}$ and $\mathbf{P}_{t-1}$ (augmented form with process noise is common in literature, but here we show the standard non-augmented version for clarity and consistency with EKF notation):

$$\chi_{t-1}^{(0)} = \hat{\mathbf{x}}_{t-1} \quad (60)$$

$$\chi_{t-1}^{(i)} = \hat{\mathbf{x}}_{t-1} + (\sqrt{(n_x + \lambda) \mathbf{P}_{t-1}})_i \quad i = 1, \dots, n_x \quad (61)$$

$$\chi_{t-1}^{(i)} = \hat{\mathbf{x}}_{t-1} - (\sqrt{(n_x + \lambda) \mathbf{P}_{t-1}})_{i-n_x} \quad i = n_x + 1, \dots, 2n_x \quad (62)$$

Propagate through the nonlinear state transition:

$$\chi_{t|i}^- = f(\chi_{t-1}^{(i)}), \quad i = 0, \dots, 2n_x \quad (63)$$

Compute predicted mean and covariance:

$$\hat{\mathbf{x}}_t^- = \sum_{i=0}^{2n_x} W_i^{(m)} \chi_{t|i}^- \quad (64)$$

$$\mathbf{P}_t^- = \sum_{i=0}^{2n_x} W_i^{(c)} (\chi_{t|i}^- - \hat{\mathbf{x}}_t^-)(\chi_{t|i}^- - \hat{\mathbf{x}}_t^-)^\top \quad (65)$$

Update Step (Measurement Update):

Propagate sigma points through the nonlinear measurement function:

$$\zeta_{t|i}^- = h(\chi_{t|i}^-), \quad i = 0, \dots, 2n_x \quad (66)$$

Compute predicted measurement mean, innovation covariance, and cross-covariance:

$$\hat{\mathbf{z}}_t^- = \sum_{i=0}^{2n_x} W_i^{(m)} \zeta_{t|i}^- \quad (67)$$

$$\mathbf{P}_{zz,t}^- = \sum_{i=0}^{2n_x} W_i^{(c)} (\zeta_{t|i}^- - \hat{\mathbf{z}}_t^-)(\zeta_{t|i}^- - \hat{\mathbf{z}}_t^-)^\top \quad (68)$$

$$\mathbf{P}_{xz,t}^- = \sum_{i=0}^{2n_x} W_i^{(c)} (\chi_{t|i}^- - \hat{\mathbf{x}}_t^-)(\zeta_{t|i}^- - \hat{\mathbf{z}}_t^-)^\top \quad (69)$$

Define the **innovation**:

$$\mathbf{v}_t = \mathbf{z}_t - \hat{\mathbf{z}}_t^- \quad (70)$$

Compute the **Kalman gain**:

$$\mathbf{K}_t = \mathbf{P}_{xz,t}^- (\mathbf{P}_{zz,t}^-)^{-1} \quad (71)$$

Update state mean and covariance:

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_t^- + \mathbf{K}_t \mathbf{v}_t \quad (72)$$

$$\mathbf{P}_t = \mathbf{P}_t^- - \mathbf{K}_t \mathbf{P}_{zz,t}^- \mathbf{K}_t^\top \quad (73)$$

B Particle flow and particle filter

B.1 Particle flow

Previously, we have discussed how after each resampling, the information of the distribution is concentrated on the multiplicity distribution of the particles, and the diversity of the particles has been lost. If we look at the particle methods, it's clear that we either use weights or multiplicity distribution to mimic the distribution. The culprit of the particle degeneracy is the consecutive multiplication of small numbers: $p(y_t|x_t^{(i)})$ will make a large number of particles irrelevant after a few steps. The observation was done independently and there is no way to control the value of $p(y_t|x_t^{(i)})$. In [11, 12], the author proposed using the multiplicity to store the information of the distribution and invented an alternative evolution for the particles such that $p(y_t|x_t^{(i)})$ was never too small. The limitation of these two papers is that to solve the equation, which we will talk about later, they have to assume a linear or nearly linear observation model and dynamics with additive Gaussian noise. The evolution for the particles is a completely different equation than the underlying dynamics for the ground truth. Later, we will see methods that generalize the equation for more general noise distributions. Let us first derive the evolution equations. First, we have the prediction step:

$$p(x_t|z_{1:t-1}) = g(x_t) \quad (74)$$

and the "observation" step,

$$p(z_t|x_t) = h(x_t) \quad (75)$$

combine both information, the updated probability for x_t should be,

$$p(x_t|z_{1:t}) \propto g(x_t)h(x_t) \quad (76)$$

The normalization will be handled implicitly by the particles. The difference between the particle flow and the particle filter is how you do the update. Since we are moving the particles directly, we demand,

$$\log p(x, \lambda) = \log g(x) + \lambda \log h(x) \quad (77)$$

we always start with $\lambda = 0$, and end with $\lambda = 1$; therefore, we can ensure the particle flows to the desired position. By definition, the evolution is driven entirely by the observation model. This is where the limitation of the flow model comes in.

Assuming particles evolve in pseudo-time λ according to the ordinary differential equation:

$$\frac{d\mathbf{x}}{d\lambda} = f(\mathbf{x}, \lambda) \quad (78)$$

where $f(\mathbf{x}, \lambda)$ is the flow field that transports particles from the prior distribution at $\lambda = 0$ to the posterior distribution at $\lambda = 1$. This deterministic evolution of particles leads to a corresponding evolution of the probability density $p(\mathbf{x}, \lambda)$.

The evolution of the probability density is governed by the Fokker-Planck equation with zero diffusion. Starting from the continuity equation for probability conservation, and noting that the flow is deterministic (no diffusion term), we obtain:

$$\frac{\partial p}{\partial \lambda} = -\nabla \cdot (pf) \quad (79)$$

This is the zero-diffusion Fokker-Planck equation, which expresses that the rate of change of probability density equals the negative divergence of the probability flux pf . To work with log-probabilities, we divide both sides by p and apply the product rule for divergence, $\nabla \cdot (pf) = p\nabla \cdot f + (\nabla p) \cdot f$, yielding: Recognizing that $\frac{1}{p} \frac{\partial p}{\partial \lambda} = \frac{\partial \log p}{\partial \lambda}$ and $\frac{\nabla p}{p} = \nabla \log p$, we obtain:

$$\frac{\partial \log p}{\partial \lambda} = -\nabla \cdot f - (\nabla \log p) \cdot f \quad (80)$$

From the log-homotopy definition, we have $\frac{\partial \log p}{\partial \lambda} = \log h(x)$, which leads to the fundamental PDE:

$$\boxed{\log h(x) = -\nabla \cdot f - (\nabla \log p) \cdot f} \quad (81)$$

To solve this equation, we assume an affine flow ansatz:

$$f(x, \lambda) = A(\lambda)x + b(\lambda) \quad (82)$$

where $A(\lambda)$ and $b(\lambda)$ are to be determined. The goal is to find expressions for these functions such that particles following this flow evolve from the prior distribution $g(x)$ at $\lambda = 0$ to the posterior distribution proportional to $g(x)h(x)$ at $\lambda = 1$.

We begin by computing the gradient of the log-homotopy. Using the Gaussian form for both $g(x)$ and $h(x)$, and substituting the explicit forms of the prior and likelihood into the log-homotopy:

$$\log p(x, \lambda) = -\frac{1}{2}(x - \bar{\eta}_0)^T P^{-1}(x - \bar{\eta}_0) - \frac{\lambda}{2}(z - Hx)^T R^{-1}(z - Hx) + \text{const} \quad (83)$$

Taking the gradient with respect to x :

$$\nabla_x \log p(x, \lambda) = -P^{-1}(x - \bar{\eta}_0) + \lambda H^T R^{-1}(z - Hx) \quad (84)$$

$$= -P^{-1}x + P^{-1}\bar{\eta}_0 + \lambda H^T R^{-1}z - \lambda H^T R^{-1}Hx \quad (85)$$

For the affine flow, the divergence is simply the trace of the matrix A :

$$\nabla \cdot f = \nabla \cdot (Ax + b) = \text{Tr}(A) \quad (86)$$

The dot product $(\nabla \log p) \cdot f$ expands as:

$$(\nabla \log p) \cdot f = [-P^{-1}(x - \bar{\eta}_0) + \lambda H^T R^{-1}(z - Hx)]^T [Ax + b] \quad (87)$$

$$= -(x - \bar{\eta}_0)^T P^{-1}Ax - (x - \bar{\eta}_0)^T P^{-1}b \quad (88)$$

$$+ \lambda(z - Hx)^T R^{-1}HAx + \lambda(z - Hx)^T R^{-1}Hb \quad (89)$$

The log-likelihood for the linear-Gaussian observation model can be expanded as:

$$\log h(x) = -\frac{1}{2}(z - Hx)^T R^{-1}(z - Hx) + \text{const} \quad (90)$$

$$= -\frac{1}{2}z^T R^{-1}z + z^T R^{-1}Hx - \frac{1}{2}x^T H^T R^{-1}Hx + \text{const} \quad (91)$$

A key insight for the EDH flow is that, under Gaussian assumptions, the distribution at any pseudo-time λ remains Gaussian. Specifically, the posterior at λ is:

$$p(x, \lambda) \sim \mathcal{N}(\bar{\eta}_\lambda, P_\lambda) \quad (92)$$

where the precision matrix and mean evolve according to:

$$P_\lambda^{-1} = P^{-1} + \lambda H^T R^{-1}H \quad (93)$$

$$\bar{\eta}_\lambda = P_\lambda[P^{-1}\bar{\eta}_0 + \lambda H^T R^{-1}z] \quad (94)$$

This allows us to express the gradient of the log-probability in a compact form:

$$\nabla \log p(x, \lambda) = -P_\lambda^{-1}(x - \bar{\eta}_\lambda) \quad (95)$$

Substituting this into the fundamental PDE, we obtain an alternative formulation:

$$\log h(x) = -\text{Tr}(A) + (x - \bar{\eta}_\lambda)^T P_\lambda^{-1}(Ax + b) \quad (96)$$

A more direct approach to determining $A(\lambda)$ and $b(\lambda)$ comes from requiring that particles starting from a Gaussian distribution remain Gaussian under the flow. For an affine flow $f = Ax + b$, the evolution of the mean and covariance must satisfy:

$$\frac{d\bar{\eta}_\lambda}{d\lambda} = A\bar{\eta}_\lambda + b, \quad \frac{dP_\lambda}{d\lambda} = AP_\lambda + P_\lambda A^T \quad (97)$$

From Kalman filter theory, we know that the precision matrix evolves as:

$$\frac{dP_\lambda^{-1}}{d\lambda} = H^T R^{-1}H \quad (98)$$

Using the identity $\frac{dP_\lambda}{d\lambda} = -P_\lambda \frac{dP_\lambda^{-1}}{d\lambda} P_\lambda$, we find:

$$\frac{dP_\lambda}{d\lambda} = -P_\lambda H^T R^{-1} H P_\lambda \quad (99)$$

Matching this with the covariance evolution equation $AP_\lambda + P_\lambda A^T = -P_\lambda H^T R^{-1} H P_\lambda$, and solving the resulting Riccati-type equation with boundary condition $P_0 = P$, we obtain:

$$A(\lambda) = -\frac{1}{2} P_\lambda H^T R^{-1} H = -\frac{1}{2} P H^T (\lambda H P H^T + R)^{-1} H \quad (100)$$

For the mean evolution, we require that $\frac{d\bar{\eta}_\lambda}{d\lambda} = H^T R^{-1} (z - H \bar{\eta}_\lambda)$ equals $A \bar{\eta}_\lambda + b$. Solving for b :

$$b(\lambda) = P_\lambda H^T R^{-1} z - A \bar{\eta}_\lambda \quad (101)$$

After algebraic manipulation, this simplifies to:

$$b(\lambda) = (I + 2\lambda A) [(I + \lambda A) P H^T R^{-1} z + A \bar{\eta}_0] \quad (102)$$

The EDH flow equations are derived by requiring that Gaussian distributions remain Gaussian under the flow, that the flow satisfies the Fokker-Planck equation, and that the boundary conditions $p(x, 0) = g(x)$ and $p(x, 1) \propto g(x)h(x)$ are satisfied. The resulting solution provides an **exact** method for transporting particles from the prior to the posterior in the linear-Gaussian case, avoiding the weight degeneracy issues that plague standard particle filters. As for the LEDH, the difference is that each particle has its own equation. That is because the expansion of the model is taken at their individual location instead of the average position.

B.2 Particle Flow Filter (Kernel Method)

Our problem is the same as for the particle filter particle flow: move the particles to the desired location, without touching the weights, so that we get the target probability distribution $p(x_t|y_{1:t}) \propto p(y_t|x_t)p(x_t|y_{1:t-1})$.

B.2.1 Optimal Transport via Gradient Flow

Define target posterior $\pi(x) := p(x_t|y_{1:t})$ and intermediate density $q_\lambda(x)$ at pseudo-time $\lambda \in [0, 1]$. With $q_0 = p(x_t|y_{1:t-1})$ (prior), $q_1 = \pi$ (posterior).

$$\frac{dx_\lambda}{d\lambda} = v_\lambda(x_\lambda) \quad (103)$$

Choose velocity v_λ to minimize KL divergence as fast as possible:

$$D_{KL}(q_\lambda \parallel \pi) = \int q_\lambda(x) \log \frac{q_\lambda(x)}{\pi(x)} dx \quad (104)$$

we need to formulate the above equation into an optimization problem. However, the intuition of this method is clear: you want to evolve the point along a "straight" direction to the desired point.

B.2.2 Deriving Optimal Velocity via Kernel Restriction

Under velocity field v , density evolves via Liouville equation:

$$\frac{\partial q_\lambda}{\partial \lambda} = -\nabla_x \cdot (q_\lambda v) \quad (105)$$

Time derivative of D_{KL} :

$$\frac{d}{d\lambda} D_{KL} = \int \frac{\partial q_\lambda}{\partial \lambda} [\log q_\lambda(x) - \log \pi(x) + 1] dx \quad (106)$$

$$= - \int \nabla_x \cdot (q_\lambda v) [\log q_\lambda(x) - \log \pi(x) + 1] dx \quad (107)$$

Using $\nabla_x q_\lambda = q_\lambda \nabla_x \log q_\lambda$ and integrating by parts:

$$\frac{d}{d\lambda} D_{KL} = - \int q_\lambda [\nabla_x \log \pi(x) \cdot v + \nabla_x \cdot v] dx \quad (108)$$

The derivative with respect to the evolution parameter gives the vector field along the trajectory. To make things computable, we restrict v to functions generated by kernel $K(x, x')$. Any such function has the form:

$$v(x) = \int \alpha(x') K(x', x) dx' \quad (109)$$

for some coefficient function $\alpha(x')$. Substitute this into directional derivative.

$$\frac{d}{d\lambda} D_{KL} = - \int q_\lambda(x) \left[\nabla_x \log \pi(x) \cdot \int \alpha(x') K(x', x) dx' + \nabla_x \cdot \int \alpha(x') K(x', x) dx' \right] dx \quad (110)$$

Interchange integrals and use integration by parts:

$$\frac{d}{d\lambda} D_{KL} = \int \alpha(x) \cdot \phi(x) dx \quad (111)$$

where

$$\phi(x) = - \int q_\lambda(x') [K(x, x') \nabla_{x'} \log \pi(x') + \nabla_{x'} K(x, x')] dx' \quad (112)$$

We have $\frac{d}{d\lambda} D_{KL} = \langle \alpha, \phi \rangle$ (inner product in function space).

To minimize this quantity (make KL decrease fastest), choose α antiparallel to ϕ . By Cauchy-Schwarz, the minimum is achieved when:

$$\alpha(x) = -\phi(x) \quad (113)$$

Therefore, optimal velocity field:

$$v_\lambda(x) = \int \alpha(x') K(x', x) dx' \quad (114)$$

$$= - \int \phi(x') K(x', x) dx' \quad (115)$$

By the reproducing property of the kernel, this simplifies to:

$$\boxed{v_\lambda(x) = \mathbb{E}_{x' \sim q_\lambda} [K(x, x') \nabla_{x'} \log \pi(x') + \nabla_{x'} K(x, x')]} \quad (116)$$

Monte Carlo approximation with particles $\{x_\lambda^{(i)}\}_{i=1}^{N_p} \sim q_\lambda$:

$$v_\lambda(x) \approx \frac{1}{N_p} \sum_{i=1}^{N_p} [K(x_\lambda^{(i)}, x) \nabla \log \pi(x_\lambda^{(i)}) + \nabla K(x_\lambda^{(i)}, x)] \quad (117)$$

Posterior gradient:

$$\nabla \log \pi(x) = \nabla \log p(y_t | x) + \nabla \log p(x | y_{1:t-1}) \quad (118)$$

First term = likelihood gradient (known). Second term = prior gradient (approximated from ensemble).

Update rule:

$$x_{\lambda+\epsilon}^{(j)} = x_\lambda^{(j)} + \frac{\epsilon}{N_p} \sum_{i=1}^{N_p} [K(x_\lambda^{(i)}, x_\lambda^{(j)}) \nabla \log \pi(x_\lambda^{(i)}) + \nabla K(x_\lambda^{(i)}, x_\lambda^{(j)})] \quad (119)$$

All particles maintain equal weight $w^{(i)} = 1/N_p$ throughout. No resampling needed.

Generality: Works for any differentiable likelihood $p(y_t | x_t)$ - not limited to Gaussian noise.

B.2.3 Scalar Kernel

Standard choice: isotropic Gaussian

$$K(x, x') = \exp \left(-\frac{1}{2\alpha\sigma^2} \|x - x'\|^2 \right), \quad \alpha \sim n_x \quad (120)$$

Gradient: $\nabla_x K(x, x') = -\frac{1}{\alpha\sigma^2} (x - x') K(x, x')$

Single scalar $K(x, x')$ weights all components identically.

B.2.4 Matrix-Valued Kernel

Measure distance independently per component:

$$\mathbf{K}(x, x') = \text{diag} \left[K^{(1)}(x, x'), \dots, K^{(n_x)}(x, x') \right] \quad (121)$$

where

$$K^{(d)}(x, x') = \exp \left(-\frac{(x^{(d)} - x'^{(d)})^2}{2\alpha(\sigma^{(d)})^2} \right), \quad \alpha = \frac{1}{N_p} \quad (122)$$

Component-wise repulsion:

$$[\nabla_x \cdot \mathbf{K}(x, x')]^{(d)} = -\frac{x^{(d)} - x'^{(d)}}{\alpha(\sigma^{(d)})^2} K^{(d)}(x, x') \quad (123)$$

Observed dimensions maintain diversity regardless of unobserved dimension spread.

References

- [1] Yudong Feng and Ashis Gangopadhyay. On a semiparametric stochastic volatility model, 2025.
- [2] Luigi Catello, Ludovica Ruggiero, Lucia Schiavone, and Mario Valentino. Hidden markov models for stock market prediction, 2023.
- [3] Olivier Guéant and Jiang Pu. Mid-price estimation for european corporate bonds: a particle filtering approach. *Market Microstructure and Liquidity*, 4(01n02):1950005, 2018.
- [4] Simo Särkkä. *Bayesian Filtering and Smoothing*. Cambridge University Press, 2013.
- [5] Shida Jiang, Junzhe Shi, and Scott Moura. A new framework for nonlinear kalman filters, 2025.
- [6] Ienkaran Arasaratnam and Simon Haykin. Cubature kalman filters. *IEEE Transactions on automatic control*, 54(6):1254–1269, 2009.
- [7] Geir Evensen. The ensemble kalman filter: Theoretical formulation and practical implementation. 2003.
- [8] Omar Al-Ghaffas, Jiajun Bao, and Daniel Sanz-Alonso. Ensemble kalman filters with resampling, 2024.
- [9] Guy Revach, Nir Shlezinger, Xiaoyong Ni, Adrià López Escoriza, Ruud J. G. van Sloun, and Yonina C. Eldar. Kalmannet: Neural network aided kalman filtering for partially known dynamics. *CoRR*, abs/2107.10043, 2021.
- [10] Hassan Mortada, Cyril Falcon, Yanis Kahil, Mathéo Clavaud, and Jean-Philippe Michel. Recursive kalmannet: Deep learning-augmented kalman filtering for state estimation with consistent uncertainty quantification. *arXiv preprint arXiv:2506.11639*, 2025. eess.SP.
- [11] Fred Daum, Jim Huang, and Arjang Noushin. Exact particle flow for nonlinear filters. In *Signal processing, sensor fusion, and target recognition XIX*, volume 7697, pages 92–110. SPIE, 2010.
- [12] Fred Daum and Jim Huang. Particle degeneracy: root cause and solution. In *Signal Processing, Sensor Fusion, and Target Recognition XX*, volume 8050, pages 367–377. SPIE, 2011.
- [13] Tao Ding and Mark J. Coates. Implementation of the daum-huang exact-flow particle filter. In *2012 IEEE Statistical Signal Processing Workshop (SSP)*, pages 257–260. IEEE, 2012.
- [14] Liyi Dai and Fred Daum. A new parameterized family of stochastic particle flow filters, 2021.
- [15] Liyi Dai and Frederick E. Daum. Stiffness mitigation in stochastic particle flow filters, 2021.
- [16] Manuel Pulido and Peter Jan van Leeuwen. Sequential monte carlo with kernel embedded mappings: The mapping particle filter. 2019.
- [17] Chih-Chi Hu and Peter Jan Van Leeuwen. A particle flow filter for high-dimensional system applications. 2021.
- [18] Sebastian Reich. A nonparametric ensemble transform method for bayesian inference. *SIAM Journal on Scientific Computing*, 35(4):A2013–A2024, 2013.
- [19] Adrien Corenflos, James Thornton, George Deligiannidis, and Arnaud Doucet. Differentiable particle filtering via entropy-regularized optimal transport, 2021.
- [20] Shreyas Chaudhari, Srinivasa Pranav, and José M. F. Moura. Gradnetot: Learning optimal transport maps with gradnets. *arXiv preprint arXiv:2507.13191*, 2025. v2.
- [21] Michael Zhu, Kevin P. Murphy, and Rico Jonschkowski. Towards differentiable resampling. *arXiv preprint arXiv:2004.11938*, 2020.
- [22] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam R. Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3744–3753. PMLR, 2019.

- [23] Xiongjie Chen and Yunpeng Li. An overview of differentiable particle filters for data-adaptive sequential bayesian inference. *Foundations of Data Science*, 2023. Also available as arXiv:2302.09639.
- [24] Brandon Amos. Tutorial on amortized optimization: Learning to optimize over continuous spaces. *arXiv preprint arXiv:2202.00665*, 2022. Version 5, October 2025.
- [25] Brandon Amos, Giulia Luise, Samuel Cohen, and Ievgen Redko. Meta optimal transport. In *Proceedings of the 40th International Conference on Machine Learning*, PMLR 202, Honolulu, Hawaii, USA, 2023. arXiv:2206.05262v2.
- [26] Yunpeng Li and Mark Coates. Particle filtering with invertible particle flow, 2017.